

AI Infrastructure Resource Ledger: The Reality Check

Thursday, 5. March 2026

Engineer(s): Carson Kohlbrenner

Date: 3/5/2026

Project Title: Predicting Collision

Phase 1: The Calendar (Working Backward)

If you are still writing your training loop on your Drop-Dead Date, your project will fail.

1. Final Demo / Presentation Date: End of April
2. Days required for Evaluation & Report: 7 days.
3. Estimated Training Run Duration: 2 days.
4. DROP-DEAD TRAINING START DATE: April 14 (*Subtract Line 2 & 3 from Line 1*)

Phase 2: The Stack & Compute Target

1. Base architecture backbone, include encoders etc., add as many lines as you need:
 - VQVAE encoder
 - Param Count: 6-12 M
 - GAT Conv spatial encoder
 - Params: 1-5 M
 - Mamba 2 dynamics model
 - Params: 10-20 M
 - Estimated total parameters: ~30M
2. Target Compute Hardware:
 - [X] Local / Lab Node
 - [] Cloud Rental (e.g., Lambda/AWS)
3. Specific GPU(s): RTX 4090
 - Total VRAM Available: 24 GB

Phase 3: The Memory Math (The VRAM Ledger)

Reference: 1B Parameters $\approx$ 2GB (Weights) + 2GB (Gradients) + 12GB (AdamW Optimizer States)

1. Model Weights (BF16): 0.06 GB
2. Gradients (BF16): 0.06 GB
3. Optimizer States: 0.36 GB
4. KV Cache & Activations Reserve: N/A GB -> Using Mamba 2 which does not use KV Caching
5. TOTAL REQUIRED VRAM: <1 GB -> Is this correct? Should I increase the scale of my project?

The Strategy: Is your Total Required VRAM > Total Available VRAM?

☐ Yes ☒ No

If Yes, what is your exact mitigation strategy? ☐ FSDP / ZeRO-3 (Sharding across multiple GPUs)

☐ LoRA / QLoRA (Freezing base weights, avoiding optimizer state explosion)

☐ I am shrinking my model selection.

☐ Something else (explain):

Phase 4: The Data Ingress (The I/O Bottleneck)

1. Dataset Size: 100-200 GB / 10000 Trajectories of 0.1-2 second videos at 60fps

2. Format:

☐ Raw JPEGs/PNGs/trajectories/etc. in a folder (High I/O Risk)

☐ WebDataset / Sharded .tar

☒ Something else -> .h5

3. Storage Location:

☒ Local NVMe SSD

☐ Network Drive

☐ S3/GCS Bucket

☐ Something else

If using raw images or other large data quantities over a network or standard SSD, how will you prevent your dataloader from starving your GPUs?

I am not sure! I am running into this problem now

Phase 5: Red Team Audit (Partner Review)

Reviewing Engineer: Yi-Shiuan Tung

Look at your partner's ledger. Find the fatal flaw. Are they underestimating training time? Do they have a 40GB model fitting into 24GB of VRAM? Are they trying to load 2 million JPEGs sequentially?

The Fatal Flaw / Biggest Risk:

Incorporating the robot's proprioceptive state into the transformer will likely lead to unpredictable hallucinations due to its high dimensionality.

Phase 6: The 72-Hour Contract

What exact, granular engineering task will you complete in the next 72 hours to unblock these risks? (e.g., "Write dummy dataloader script to verify throughput," "Run a 10-step overfitting batch to check VRAM usage.")

Train on a single trajectory with action and proprioception, and compare to the static result.

Deliverable by Monday: Include actions in the dynamics model.